

PDFiTT: Open-Source PDF Generation MCP

Generated: April 29, 2026

Executive Summary

PDFiTT is an open-source, self-hostable PDF generation MCP for turning Markdown into polished PDF documents. It provides a browser-based editor for writing and previewing Markdown, a direct HTTP PDF API, a remote Streamable HTTP MCP server, and a portable local agent skill for Codex, OpenCode, Claude Code, and other MCP-capable runtimes.

The repository is structured so a human or an LLM agent can be pointed at the repo and quickly understand how to install, configure, deploy, validate, and customize a hosted PDF generation tool.

The system is intentionally small:

- The frontend is a React and Vite single-page application.
- Markdown parsing is handled by `marked`.
- PDF generation is handled by `pdfmake`.
- Shared Markdown-to-PDF conversion logic lives under `src/lib`.
- Vercel Functions expose `/api/pdf` and `/api/mcp`.
- Vercel rewrites `/mcp` to `/api/mcp`.
- The local agent skill wraps the remote MCP endpoint with a deterministic Node.js script.
- The project is distributed under the MIT License.
- LLM-oriented setup entry points are provided in `llms.txt` and `AGENTS.md`.

Positioning

PDFiTT should be described as:

An open-source, self-hostable PDF generation MCP that renders Markdown into formatted PDFs through a remote Streamable HTTP MCP endpoint, direct API, browser UI, and portable agent skill.

Short description:

Open-source Markdown-to-PDF generation for agents, apps, MCP clients, and self-hosted PDF tools.

Primary differentiators:

- Open-source implementation under the MIT License.
- Remote MCP endpoint that does not require a specific app framework or model provider.
- Direct HTTP API for non-MCP clients.
- Browser UI for human-authored Markdown.
- Shared renderer across browser, API, and MCP flows.
- Portable local skill for agent workflows.
- Root-level documentation designed for both humans and LLM agents.

LLM And Human Onboarding

PDFiTT includes root-level documentation intended to make repository setup deterministic for both humans and LLM agents:

File	Audience	Purpose
README.md	Humans and agents	Primary overview, quick start, API examples, MCP setup, and self-host summary.
llms.txt	LLM agents	Machine-oriented project map with exact install, deploy, MCP, and validation steps.
AGENTS.md	LLM coding agents	Repo-specific implementation rules, validation commands, and release checklist.
SELF_HOSTING.md	Humans and agents	Complete local setup, Vercel

File	Audience	Purpose
		deployment, environment variables, verification, and customization guide.
CHANGELOG.md	Humans and release tooling	Release history.
docs/PDFiTT-System-Documentation.md	Humans and agents	Full architecture and operations reference.
docs/PDFiTT-System-Documentation.pdf	Humans	Rendered system documentation PDF.

System Goals

PDFiTT is designed to:

- Convert Markdown into a polished PDF without requiring a document template.
- Provide the same rendering behavior in browser, direct API, and MCP workflows.
- Allow agents to render local Markdown safely by reading local files themselves and sending Markdown content to the remote MCP server.
- Keep deployment simple by using Vercel Functions and a static Vite frontend.
- Avoid executing raw HTML or script content from user-provided Markdown.
- Make self-hosting straightforward for developers who want their own PDF generation MCP.
- Give LLM agents enough repo-local instructions to install, deploy, test, and customize without external context.

Repository Layout

PDFiTT/ AGENTS.md CHANGELOG.md

SELF_HOSTING.md
llms.txt
api/
mcp.js Remote MCP endpoint.
pdf.js Direct PDF HTTP endpoint.
docs/
PDFiTT-System-Documentation.md
PDFiTT-System-Documentation.pdf
public/
PDFiTT_favicon.ico
llms.txt Hosted LLM entry point copied into the Vite build.
skills/
pdfitt-markdown-pdf/
SKILL.md Agent skill instructions.
agents/
openai.yaml
opencode.json
scripts/
render_pdf_via_mcp.mjs
src/
App.jsx Browser editor and preview UI.
index.css
main.jsx
lib/
markdownPdf.js Markdown token conversion and PDF document definition.
pdfBuffer.js Server-side PDF buffer rendering and input resolution.
index.html
package.json
package-lock.json
tailwind.config.js
vercel.json
vite.config.js

Runtime Architecture

Browser UI
|
| Markdown text
v
React App.jsx

```
|
| buildDocDefinition(markdown)
v
pdfmake in browser
|
v
Downloaded PDF

Direct API client
|
| POST /api/pdf
v
Vercel Function api/pdf.js
|
| resolveMarkdownInput -> renderMarkdownPdfBuffer
v
PDF response

MCP client or agent skill
|
| JSON-RPC POST /mcp tools/call render_markdown_pdf
v
Vercel Function api/mcp.js
|
| resolveMarkdownInput -> renderMarkdownPdfBuffer
v
Base64 PDF payload
```

Main Components

Frontend Application

The frontend entry point is `src/App.jsx`. It provides:

- A Markdown editor backed by React state.
- A live HTML preview rendered with `marked`.
- HTML sanitization through `DOMPurify`.
- A word-count display.

- Reset and clear actions.
- A client-side PDF download button.

The browser workflow uses `buildDocDefinition(text)` from `src/lib/markdownPdf.js`, dynamically loads `pdfmake`, then calls `pdfMake.createPdf(...).download(...)`.

Markdown Conversion Library

`src/lib/markdownPdf.js` is the central conversion module. It exports:

- `MAX_MARKDOWN_LENGTH`
- `cleanText`
- `getDocumentTitle`
- `sanitizeFileName`
- `getWordCount`
- `buildDocDefinition`
- `validateMarkdown`

It parses Markdown with `marked.lexer` using GitHub Flavored Markdown mode, then converts the resulting token stream into a `pdfmake` document definition.

Supported Markdown features include:

- H1 through H4 headings.
- Paragraphs.
- Ordered and unordered lists.
- Task list items.
- Blockquotes.
- Fenced code blocks.
- Inline code.
- Links.
- Images as text placeholders.
- Tables.
- Horizontal rules.
- Bold, italic, and strikethrough inline spans.

- Basic HTML-to-text fallback.

The generated PDF uses:

- Letter page size.
- Roboto font.
- 54-point horizontal margins.
- 56-point vertical margins.
- Title and page-count footer on every page.
- Structured styles for headings, paragraphs, code, lists, and tables.

Server-Side PDF Buffer Library

`src/lib/pdfBuffer.js` adapts the shared Markdown conversion code for Vercel Functions and MCP usage. It exports:

- `resolveMarkdownInput`
- `renderMarkdownPdfBuffer`
- `getPdfFileName`

`resolveMarkdownInput` accepts exactly one logical Markdown source:

- `markdown`: direct Markdown string.
- `markdown_base64`: base64-encoded UTF-8 Markdown.
- `source_url`: public `http` or `https` URL.

For local files, callers should read the file locally and pass the Markdown content as `markdown`. The remote service cannot read private local filesystem paths.

`renderMarkdownPdfBuffer` validates Markdown and then creates a PDF buffer with `pdfmake`.

`getPdfFileName` derives a safe `.pdf` filename from a requested filename or the first H1 heading.

Public Interfaces

Browser Interface

The browser interface is served from the Vite build output. Users paste or write Markdown, inspect the preview, and download the PDF.

Direct PDF API

Endpoint:

```
POST /api/pdf
```

Deployed URL:

```
https://pdf-i-tt.vercel.app/api/pdf
```

Request JSON:

```
{
  "markdown": "# Hello\n\nRendered by PDFiTT.",
  "filename": "hello.pdf"
}
```

Alternative inputs:

```
{
  "markdown_base64": "lyBIZWxsby4="
}
```

```
{
  "source_url": "https://example.com/document.md"
}
```

Successful response:

- Status: 200

- Content-Type: application/pdf
- Content-Disposition: attachment; filename="...".
- Body: PDF bytes.

Common error responses:

- 400 for invalid JSON, invalid input, unsupported URL schemes, failed source URL fetches, empty Markdown, or oversized Markdown.
- 405 for methods other than POST and OPTIONS.
- 403 for disallowed origins.

Remote MCP Endpoint

Endpoint:

```
POST /mcp
```

Deployed URL:

```
https://pdf-i-tt.vercel.app/mcp
```

Transport:

```
streamable_http
```

Protocol version:

```
2025-06-18
```

Server info:

```
{  
  "name": "pdfitt-markdown-pdf",  
  "version": "1.0.0"
```

```
}
```

Tools:

- `render_markdown_pdf`
- `get_setup_instructions`

The endpoint is stateless. It supports JSON-RPC POST requests and responds to `notifications/initialized` with HTTP 202.

GET behavior:

- A normal GET returns setup instructions as JSON.
- A GET with `Accept: text/event-stream` receives a 405 response because this endpoint supports POST-only stateless MCP.

MCP Tool: `render_markdown_pdf`

Purpose:

Render Markdown into a formatted PDF and return the PDF as base64.

Input schema:

```
{
  "type": "object",
  "properties": {
    "markdown": { "type": "string" },
    "markdown_base64": { "type": "string" },
    "source_url": { "type": "string" },
    "filename": { "type": "string" }
  },
  "additionalProperties": false
}
```

Output payload:

```
{
```

```
"filename": "document.pdf",  
"mimeType": "application/pdf",  
"byteLength": 22095,  
"base64": "JVBERi0x..."  
}
```

The payload appears in both `content[0].text` as JSON text and `structuredContent` as structured JSON.

MCP Tool: `get_setup_instructions`

Purpose:

Return connection instructions for generic MCP clients, Claude Code, OpenCode, Codex local skill installation, local skill script usage, and the direct PDF API.

Output:

- Human-readable instructions in `content[0].text`.
- Structured setup metadata in `structuredContent`.

Agent Skill

The portable skill lives at:

```
skills/pdfitt-markdown-pdf
```

Installed for Codex at:

```
~/codex/skills/pdfitt-markdown-pdf
```

The skill tells agents to use the remote MCP endpoint when available and to avoid sending private local paths to the remote server. Local files should be read by the agent first, then their Markdown content should be sent through MCP.

The deterministic script is:

```
skills/pdfitt-markdown-pdf/scripts/render_pdf_via_mcp.mjs
```

Example:

```
node skills/pdfitt-markdown-pdf/scripts/render_pdf_via_mcp.mjs \  
--mcp-url https://pdf-i-tt.vercel.app/mcp \  
--markdown-file ./notes.md \  
--out ./notes.pdf
```

The script performs:

1. initialize
2. notifications/initialized
3. tools/call for render_markdown_pdf
4. Base64 PDF decoding
5. Local PDF file write

Input Validation and Limits

Markdown validation rules:

- Input must be a string.
- Input cannot be empty after whitespace normalization.
- Input cannot exceed 250,000 characters.

Source URL rules:

- source_url must use http or https.
- The remote server fetches the source URL.
- Non-OK HTTP responses produce an error.

Filename rules:

- The first H1 heading is used as the default document title.
- If no H1 is present, the title defaults to PDFiTT Document.

- Filenames are normalized to lowercase.
- Non-alphanumeric runs are replaced with hyphens.
- Filenames are capped at 80 characters before `.pdf`.
- Empty sanitized filenames fall back to `pdfitt-document.pdf`.

Security Model

PDFiTT does not execute user-provided Markdown. Important controls include:

- Browser preview HTML is sanitized with `DOMPurify`.
- Raw `<script>` and `<style>` blocks are stripped from PDF text conversion.
- Other HTML tags are converted to plain text.
- `source_url` only allows `http` and `https`.
- Remote MCP cannot access local files.
- CORS origin checks use `PDFITT_ALLOWED_ORIGINS` when configured.
- Requests from the same host, `localhost`, or `127.0.0.1` are allowed by origin logic.

The MCP endpoint also validates optional request headers:

- `Mcp-Method` must match the JSON-RPC method when present.
- `Mcp-Name` must match the tool name for `tools/call` when present.

Environment Variables

PDFITT_PUBLIC_URL

Used by `api/mcp.js` to build setup URLs.

Default:

```
https://pdf-i-tt.vercel.app
```

PDFITT_ALLOWED_ORIGINS

Comma-separated list of allowed browser origins for API and MCP requests.

Example:

```
https://pdf-i-tt.vercel.app,https://example.com
```

When unset, same-host requests and local development origins are still allowed.

PDFITT_MCP_URL

Used by the local skill script as an alternative to the `--mcp-url` flag.

Example:

```
PDFITT_MCP_URL=https://pdf-i-tt.vercel.app/mcp \  
node skills/pdfitt-markdown-pdf/scripts/render_pdf_via_mcp.mjs \  
--markdown-file ./notes.md \  
--out ./notes.pdf
```

LLM Agent Setup Contract

When an LLM agent is pointed at this repository, it should be able to perform the following without additional context:

1. Read `llms.txt`.
2. Read `README.md`, `AGENTS.md`, and `SELF_HOSTING.md`.
3. Install dependencies with `npm install`.
4. Run `npm run lint` and `npm run build`.
5. Deploy the app to Vercel or explain that deployment credentials are missing.
6. Set `PDFITT_PUBLIC_URL` on the hosted deployment.
7. Verify `GET /mcp`, `POST /api/pdf`, and MCP `render_markdown_pdf`.
8. Replace hosted examples with the user's own deployed origin when configuring MCP clients.

Agent-safe rules:

- Do not send local filesystem paths to a remote MCP server.
- Read local Markdown files locally and pass the file contents as `markdown`.
- Use `source_url` only for public HTTP or HTTPS Markdown URLs.
- Keep examples copy-pasteable.
- Update `llms.txt`, `AGENTS.md`, `SELF_HOSTING.md`, and this system document when setup behavior changes.

Deployment

The project deploys to Vercel.

`vercel.json` configures:

- `/mcp` rewrite to `/api/mcp`.
- 30-second maximum duration for `api/mcp.js`.
- 30-second maximum duration for `api/pdf.js`.

Build commands come from `package.json`:

```
npm run build
```

Local development:

```
npm install  
npm run dev
```

Self-hosting flow:

1. Fork or clone the repository.
2. Install dependencies with `npm install`.
3. Run locally with `npm run dev`.
4. Deploy to Vercel.
5. Set `PDFITT_PUBLIC_URL` to the deployed origin, for example `https://your-pdf-tool.vercel.app`.

- 6. Optionally set `PDFiTT_ALLOWED_ORIGINS` to a comma-separated allowlist.
- 7. Connect MCP clients to `https://your-pdf-tool.vercel.app/mcp`.
- 8. Use the direct API at `https://your-pdf-tool.vercel.app/api/pdf`.

Self-hosted verification:

```
curl https://your-pdf-tool.vercel.app/mcp
```

```
curl -X POST "https://your-pdf-tool.vercel.app/api/pdf" \
-H "Content-Type: application/json" \
--output /tmp/pdfitt-api-test.pdf \
--data '{"markdown":"# PDFiTT API Test\n\nRendered from a self-hosted deployment."}'
```

```
node skills/pdfitt-markdown-pdf/scripts/render_pdf_via_mcp.mjs \
--mcp-url https://your-pdf-tool.vercel.app/mcp \
--markdown "# PDFiTT MCP Test" \
--out /tmp/pdfitt-mcp-test.pdf
```

Quality checks:

```
npm run lint
npm run build
npm audit
```

Dependencies

Runtime dependencies:

Package	Purpose
react	Frontend UI runtime.
react-dom	Browser rendering for React.

Package	Purpose
vite	Frontend build and dev server.
marked	Markdown parsing and tokenization.
pdfmake	PDF document generation.
dompurify	Browser preview sanitization.
lucide-react	UI icons.
tailwindcss	Utility CSS framework.

Development dependencies include ESLint, React ESLint plugins, PostCSS, Autoprefixer, Vite React plugin, and TypeScript type packages for React.

Data Flows

Browser Download Flow

1. User edits Markdown in the textarea.
2. React stores Markdown in local component state.
3. Preview HTML is generated with `marked.parse`.
4. Preview HTML is sanitized with `DOMPurify`.
5. User clicks Download PDF.
6. Browser dynamically imports pdfmake and fonts.
7. `buildDocDefinition` creates a pdfmake document definition.
8. pdfmake downloads the PDF in the browser.

Direct API Flow

1. Client sends JSON to `/api/pdf`.
2. `api/pdf.js` checks origin and HTTP method.

3. Request body is parsed.
4. `resolveMarkdownInput` resolves direct, base64, or URL Markdown.
5. `renderMarkdownPdfBuffer` validates and renders the PDF.
6. Response returns PDF bytes with download headers.

MCP Flow

1. MCP client calls `initialize`.
2. Server returns capabilities, server info, setup instructions, and tool capability.
3. MCP client sends `notifications/initialized`.
4. MCP client calls `tools/call` with `render_markdown_pdf`.
5. Server resolves Markdown input and renders PDF bytes.
6. Server returns filename metadata and base64 PDF.
7. Client decodes base64 and writes or attaches the PDF.

Operational Notes

- Keep Markdown under 250,000 characters.
- Prefer direct `markdown` input for private local files.
- Use `source_url` only for public Markdown URLs.
- Restart MCP clients after adding the remote server if their runtime does not hot-reload MCP config.
- If rendering fails in an agent, run the bundled script directly to isolate MCP transport and PDF rendering.
- If direct API calls fail from a browser, check CORS origin configuration first.

Troubleshooting

MCP server is not visible in Codex

Check registration:

```
codex mcp list
codex mcp get pdfitt
```

Expected URL:

```
https://pdf-i-tt.vercel.app/mcp
```

Restart Codex if the MCP server was added during an active session.

PDF generation fails with "Markdown cannot be empty"

The input string is empty or contains only whitespace. Provide non-empty Markdown.

PDF generation fails with "Markdown is too large"

Reduce the Markdown body below 250,000 characters or split the document into multiple PDFs.

PDF generation fails with source URL errors

Confirm that:

- The URL is public.
- The URL uses `http` or `https`.
- The server returns a successful 2xx response.
- The response body is Markdown text.

Browser preview differs from PDF output

The browser preview renders sanitized HTML, while PDF output is produced from Markdown tokens. Raw HTML and scripts are intentionally reduced to plain text or removed for PDF safety.

Maintenance Checklist

Before release:

- Run `npm run lint`.
- Run `npm run build`.
- Test direct API rendering.
- Test MCP `tools/list`.
- Test MCP `render_markdown_pdf`.
- Test local skill script rendering.
- Check CORS behavior for expected production origins.
- Confirm `/mcp` GET returns current setup instructions.

When changing Markdown rendering:

- Update `src/lib/markdownPdf.js`.
- Confirm browser and server outputs still share the same behavior.
- Render a sample document containing headings, lists, tables, links, blockquotes, code blocks, inline code, and task lists.
- Verify long words wrap without breaking page layout.

When changing MCP behavior:

- Update `api/mcp.js`.
- Keep `render_markdown_pdf` and `get_setup_instructions` schemas synchronized with `skills/pdfitt-markdown-pdf/SKILL.md`.
- Validate JSON-RPC error handling.
- Confirm the local script still handles initialize, initialized notification, and tool call responses.

Current Installed MCP Configuration

The local Codex MCP server registration is:

```
Name: pdfitt
Transport: streamable_http
URL: https://pdf-i-tt.vercel.app/mcp
Status: enabled
```

The local Codex skill installation is:

```
~/codex/skills/pdfitt-markdown-pdf
```

Appendix: Example JSON-RPC MCP Call

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/call",
  "params": {
    "name": "render_markdown_pdf",
    "arguments": {
      "markdown": "# Example\n\nRendered by PDFiTT.",
      "filename": "example.pdf"
    }
  }
}
```

Appendix: Direct API Example

```
curl -X POST "https://pdf-i-tt.vercel.app/api/pdf" \
-H "Content-Type: application/json" \
--output document.pdf \
--data '{"markdown":"# Hello\n\nRendered by PDFiTT."}'
```

Appendix: Local Skill Script Example

```
node ~/codex/skills/pdfitt-markdown-pdf/scripts/render_pdf_via_mcp.mjs \
--mcp-url https://pdf-i-tt.vercel.app/mcp \
--markdown-file ./docs/PDFiTT-System-Documentation.md \
--out ./docs/PDFiTT-System-Documentation.pdf
```